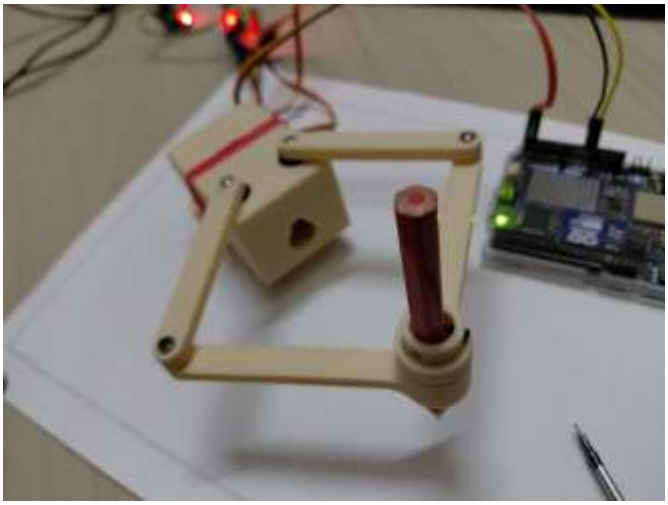
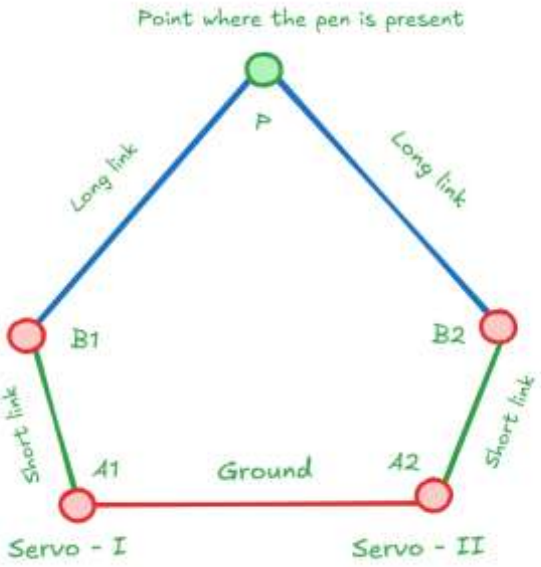
Five-Link Parallel Drawing Robot

Forward kinematics, inverse kinematics, the Jacobian, singularities, and a Jacobian-transpose solver

How to read this: every section first says what question it answers, then the recipe in plain words, then the formula. The symbols are the same as in the code (Solver.py), so you can read the two side by side. The last section lists what carries over unchanged to a delta robot.

1. The robot





Left: the printed robot. Right: the names used everywhere in this document and in the code.

The robot has four moving links and one fixed one:

- **Two short links** (length a = 60 mm), each bolted to a servo. The servo sets the angle of its short link. These are the only two things the motors control.
- **Two long links** (length b = 70 mm), each pinned to the end of a short link at a free joint, the elbow. The two long links meet at the pen.

- **The ground**: the servo box itself, which holds the two servo shafts d = 25.8 mm apart. It never moves, but it closes the loop, so it counts as the fifth link. That is why this is called a five-bar linkage.

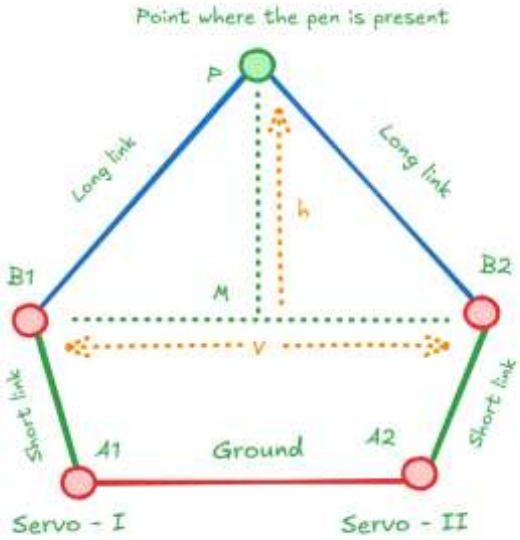
Because the links form a closed loop (ground, short, long, long, short, back to ground) the maths is different from a normal two-link arm, where the links are chained one after the other. The main differences show up in the Jacobian and in the singularities later on.

Frame. The origin O is halfway between the two servo shafts. x points to the robot's right, y points toward the paper. Angles are measured counter-clockwise from +x. With this choice the servos sit at A1 = (−d/2, 0) = (−12.9, 0) and A2 = (+d/2, 0) = (12.9, 0).

Names: A1 and A2 are the servo shafts (fixed). B1, B2 are the elbows. P = (x, y) is the pen. θ1, θ2 are the short-link angles set by the servos. Everything else is derived from these.

2. Forward kinematics (FK): angles → pen position

Question answered: if the servos are at θ1 and θ2, where is the pen?



The isosceles triangle B1 P B2 with base V, midpoint M and height h is the whole FK calculation.

Recipe

1. Elbows. Each short link is a stick of length a pinned at its servo, so its far end is at $B1 = A1 + a\cdot(\cos\theta1, \sin\theta1)$ and $B2 = A2 + a\cdot(\cos\theta2, \sin\theta2)$.
2. Elbow-to-elbow arrow. $V = B2 - B1$, with length $L = |V|$.
3. Feasibility check. The two long links are 70 each, so the elbows can be at most $2b = 140$ apart. If $L > 140$ the loop cannot close and the angles are impossible.
4. Midpoint. $M = (B1 + B2) / 2$. (Note: this is the average of the two points, not V/2. V/2 is an arrow, M is a point.)
5. Height. The pen is 70 from both elbows, so B1, P, B2 is an isosceles triangle. Pythagoras in one half of it gives $h = \sqrt{b^2 - (L/2)^2}$. Note the minus sign: the long link is the hypotenuse.
6. Perpendicular direction. Rotate V by +90°: $n = (-Vy, Vx) / L$. Dividing by L makes it length 1, so it carries direction only.
7. Pen. $P = M + h\cdot n$.

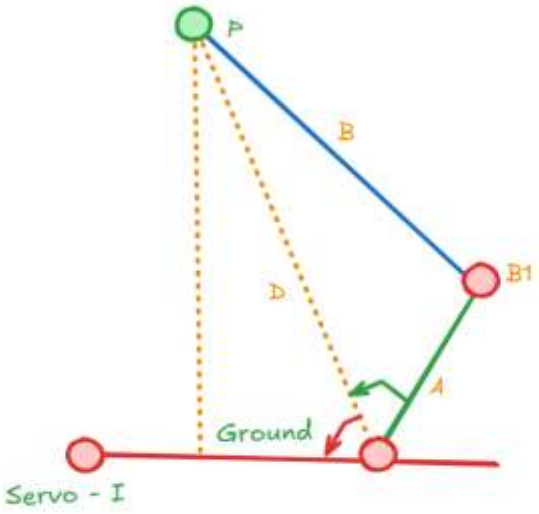
Why + and not −. Two circles of radius 70 around the elbows cross at two points, one on the paper side (M + h·n) and one on the base side (M − h·n). The physical robot was assembled with the pen on the paper side, so we always take +. The maths alone cannot tell the two apart; this is a choice we make once.

Check. At θ1 = 120°, θ2 = 60° this gives P = (0, 107.28). Both |P − B1| and |P − B2| must come back as 70.00.

3. Inverse kinematics (IK): pen position → angles

Question answered: I want the pen at (x, y). What do I send to the servos?

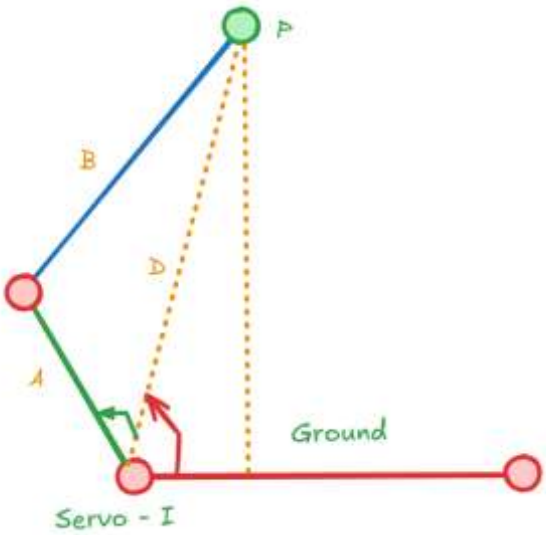
Key idea. The two sides do not interact. The left side is a two-link chain from the fixed point A1 to the known point P; the right side is the same from A2. Solve each side on its own with the same recipe.



One side: triangle A–B–P. All three sides are known (a, b, and the servo-to-pen distance D), so the angle at the servo follows from the law of cosines.

Recipe (for one side, servo at A)

1. Arrow from servo to pen: $(\Delta x, \Delta y) = P - A$. Its length is $D = \sqrt{(\Delta x^2 + \Delta y^2)}$.
2. Reach check: the triangle only closes if $|a - b| \le D \le a + b$, i.e. $10 \le D \le 130$. Otherwise the point is unreachable for this side.
3. Direction of that arrow: $\varphi = \text{atan2}(\Delta y, \Delta x)$ (the red angle in the picture). Use atan2, not atan(Δy/Δx): it handles Δx = 0 and all four quadrants.
4. Opening angle between the arrow A→P and the short link (the green angle): law of cosines, $\cos \alpha = (a^2 + D^2 - b^2) / (2 \cdot a \cdot D)$, then $\alpha = \arccos(\ldots)$. The formula gives the cosine, so the arccos is needed to get the angle.
5. Put them together: the short link points at $\varphi \pm \alpha$.

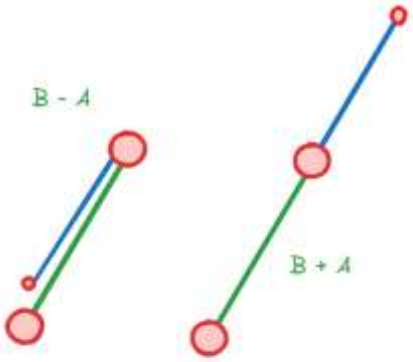


Left servo: θ1 = φ + α. Right servo: θ2 = φ − α. The sign chooses which side the elbow sits on.

Why + on the left and − on the right. ±α are the two mirror-image elbow positions (elbow-out and elbow-in). The printed robot runs elbow-out: the left elbow swung further counter-clockwise (+α), the right elbow further clockwise (−α). Elbow-in would make the two short links cross each other, which the plastic does not allow.

Loop check. Always feed the IK answer back into FK. For P = (30, 90): IK gives θ1 = 108.28°, θ2 = 29.42°; FK of those angles returns (30.00, 90.00). If it does not, there is a sign or quadrant mistake.

4. Reach limits

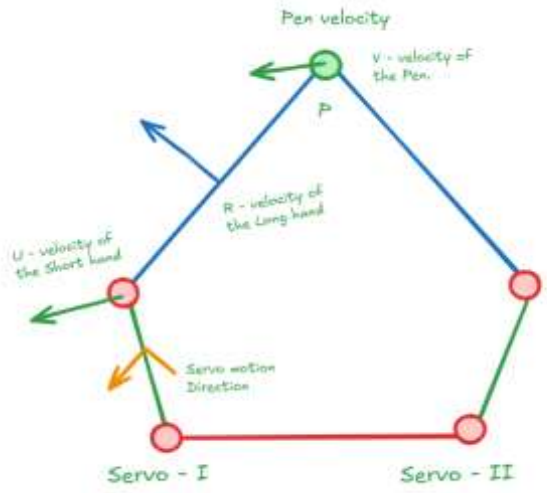


One side can reach points between b − a (folded) and b + a (stretched) from its servo.

- **FK side:** L ≤ 2b = 140. The elbows cannot be further apart than two long links laid end to end.
- **IK side:** b − a ≤ D ≤ a + b, i.e. 10 ≤ D ≤ 130 from each servo. Each side can reach a ring (annulus) around its servo; the pen must be inside both rings.
- **On top of that:** the elbow-out assembly must hold, and both servo commands must be inside 0–180°. The usable drawing area is smaller than the geometric one; for this robot roughly x in [−40, 40], y in [45, 115].

5. The Jacobian: how fast does the pen move?

Question answered: if servo 1 turns at ω1 and servo 2 at ω2 (rad/s), how fast and in which direction does the pen move right now? The answer is a 2×2 matrix J with v = J·ω. It changes from pose to pose.



U is the elbow velocity (from the servo), R is the long-link arrow, v is the pen velocity we want.

Step 1: how the elbow moves. The elbow runs on a circle of radius a around its servo, so its velocity is at right angles to the short link, with speed a·ω. In vector form: U1 = a·ω1·(−sin θ1, cos θ1). (This is the time-derivative of B1 = A1 + a·(cos θ1, sin θ1); the chain rule brings in ω1 = dθ1/dt.)

Step 2: what a rigid long link allows. The pen is the elbow plus the long-link arrow R1 = P − B1. So v = U1 + (rate of change of R1). R1 has fixed length 70, and an arrow of fixed length can only rotate, so its rate of change is at right angles to R1. We do not know its size and we never need it.

Step 3: remove the unknown with a dot product. Dot both sides with R1. The unknown rotation part is at right angles to R1, so its dot product is zero and it disappears:

$$R1 \cdot v = R1 \cdot U1 = a \cdot \omega 1 \cdot [\, R1 \cdot (-\sin\theta 1, \cos\theta 1) \,] = c1 \cdot \omega 1$$

In words: the pen and the elbow move by the same amount along the long link (a rigid stick cannot stretch); across the link they may differ (the link swings). The number c1 = a·[R1·(−sin θ1, cos θ1)] is how much of the elbow's motion lands along long link 1. It is one number per pose; call it the transmission number.

Step 4: the same for side 2, then stack. Side 2 gives R2 · v = c2 · ω2. Two equations, two unknowns (vx, vy):

$$\begin{bmatrix} R1 \\ R2 \end{bmatrix} \cdot v = \begin{bmatrix} c1 & 0 \\ 0 & c2 \end{bmatrix} \cdot \omega \qquad \text{i.e.} \quad LHS \cdot v = RHS$$

LHS = rows are the two long-link arrows (in the code: LHS)
diag(c1, c2) = the two transmission numbers (in the code: c1, c2 inside RHS)

Solving for v gives $v = LHS^{-1} \cdot \mathrm{diag}(c1, c2) \cdot \omega$, so

$$J = LHS^{-1} \cdot \mathrm{diag}(c1, c2) \qquad v = J \cdot \omega$$

Reading J. Column 1 is the pen velocity when only servo 1 turns at 1 rad/s; column 2 the same for servo 2. For any ω the pen velocity is column1·ω1 + column2·ω2. (Note: the 2×2 matrix on the left of the equations is LHS, not J. J only appears after the solve.) At P = (30, 90):

```
J = [ −51.14  −28.32 ]   mm/s per rad/s        1° on servo 1 alone moves the pen (−0.89, −0.52) mm, about 1 mm.
    [ −29.71   52.93 ]
```

Backwards: servo speeds for a wanted pen velocity. Each equation has only its own ω, so no matrix inverse is needed:

```
ω1 = (R1 · v) / c1        ω2 = (R2 · v) / c2
```

"Project the wanted pen velocity onto my long link, divide by my transmission number." For v = (100, 0) mm/s at (30, 90): ω1 = −85.5 °/s, ω2 = −48.0 °/s. An SG90 manages roughly 400–600 °/s, so this is comfortable. This is what a controller runs at every step of a line.

Why this is different from a serial arm. For a chained arm, FK is a plain formula, and J is its derivative, one column per joint. Here the pen is where two circles cross, so instead of differentiating FK we differentiate the constraint "pen is 70 from each elbow". That produces two matrices (LHS and diag(c)) instead of one, which is exactly why there are two kinds of singularity below. J is still the derivative of FK; nudging θ1 by a tiny amount and measuring the pen's move gives column 1 exactly.

6. Singularities

What a singularity is. J is an exchange rate: pen speed per servo speed. A singularity is a pose where that exchange rate breaks, becoming zero or infinite. The clean way to see where this happens is the determinant. Because J is a product, its determinant is a fraction:

```
det(J) = (c1 · c2) / det(LHS)        write D = det(LHS)
```

A fraction can go wrong in two ways: the top can be zero, or the bottom can be zero. These are the two singularity types, and they feel completely different on the real robot.

6.1 Type 1, the wall: c1 = 0 or c2 = 0 (short link and long link in one line)

c1 = a·[R1·(−sin θ1, cos θ1)] is a dot product between the long link and the direction the elbow moves in. It is zero when they are at right angles. The elbow already moves at right angles to the short link, so this means the short link and the long link are in one straight line: the side is fully stretched (D = 130) or fully folded (D = 10).

What happens. All of the elbow's motion is sideways to the long link, none of it pushes along it. Servo 1 can swing the straight side like a stick, but it cannot push the pen any further out. One direction of pen motion is lost. Column 1 of J becomes (0, 0) and det(J) → 0. If you ask for pen motion along that link anyway, the servo speed ω1 = (R1·v)/c1 is a division by zero: infinite.

What you feel. Nothing special: the robot is rigid, like your own elbow locked straight. You simply cannot extend it further. With the servos powered you cannot push the pen in any direction here.

Where it is. Exactly on the edge of the workspace (the IK reach check D ≤ a + b is the same condition). It is harmless if you stay inside; near it, the last millimetres toward the edge cost a lot of servo motion.

6.2 Type 2, the trap: D = det(LHS) = 0 (the two long links in one line)

det(LHS) is built from the two long-link arrows R1 and R2 only. For a 2×2 matrix the determinant is the area of the parallelogram spanned by the two rows: D = 70·70·sin(angle between the long links). It is zero when R1 and R2 are parallel: B1, P and B2 lie on one line, the FK triangle has collapsed flat (h = 0, elbows 140 apart). For this geometry it happens at y ≈ 18.4 mm.

What happens. The two equations now pin the pen's motion along the same line twice and say nothing about the direction across it. LHS cannot be inverted and J blows up (det(J) → ∞). Both long links pull on the pen only along their common line, so nothing holds the pen at right angles to it.

What you feel. This is the strange one: with both servos powered and holding, the pen can be pushed up and down with a fingertip. Moving the pen 1 mm across the flat links asks each link to stretch by only $1^2/140 = 0.007$ mm, which the plastic and the pin clearance absorb. The elbows do not move, the servo shafts do not turn, so the servos never see an error and never resist. Push past the line and the linkage flips to the other assembly (pen on the base side); every FK/IK answer afterwards is for the wrong assembly.

Where it is. Below the drawing area, near the servos. The firmware's y ≥ 25 limit exists to stay clear of it. On the real robot it feels mushy before it feels loose: a couple of degrees away from flat the servos still resist through friction and a large lever ratio; the pen only goes truly free in the last millimetre.

6.3 The two types side by side

	Type 1: wall	**Type 2: trap**
Geometry	short link and long link of one side in a line	the two long links in a line through the pen
Number that dies	c1 or c2 → 0, so det(J) → 0	D = det(LHS) → 0, so det(J) → ∞
Pen	loses a direction (cannot go further out)	gains a free direction (moves with servos locked)
Servo speed for a given pen speed	→ ∞ along the straight link	→ 0 in the free direction (servos not involved)
Feels like	rigid wall, elbow locked straight	tight rope: loose in the middle, can flip
Where	edge of the workspace (same as reach limit)	inside the geometric workspace, near the base
Danger	low: IK refuses points beyond	high: bowed links, assembly flip, jittery control
Exists on a serial arm?	yes (elbow straight)	no: only parallel robots have it

6.4 Near, not at: three numbers to watch

Trouble starts well before the exact pose. Three numbers can be computed at every pen position from the same Jacobian() call:

- $|D| = |\det(\text{LHS})|$: small means the long links are nearly in line (type 2). At (30, 90) it is 4897; at (0, 25) it is already 814.
- $|\det(J)|$ (or directly $\min(|c1|, |c2|)$): small means a side is nearly straight (type 1). Low near the top of the workspace.
- $\text{cond}(J)$, the ratio of the longest to the shortest axis of the pen-motion ellipse for 1° steps: 1 is ideal, large means one direction is poorly controlled. It rises toward either singularity.

There is no universal threshold; the raw determinants are in mm² and depend on the link lengths. The right way to pick them is to work backwards from the hardware: decide a pen speed, a servo speed limit with a safety factor, and an acceptable step size, run the table over the workspace, and read off the D and c values at the boundary where a test fails. For this robot and 100 mm/s that gives roughly $|D| > 1500$ and $|c| > 1000$; below that the servo speed demand and the step size per degree grow quickly. Re-do the table whenever a link length, the servo, or the pen speed changes.

In code: before using J^{-1} or the servo-speed formula, test $|D|$ and $|ci|$ against the thresholds and refuse the point or damp the step. After any type-2 scare, run FK on the commanded angles and check that the pen is above the elbow line.

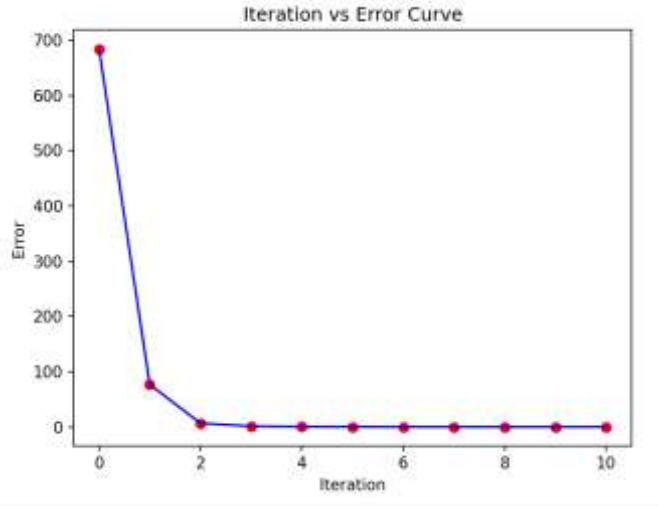
7. Solver: inverse kinematics by gradient descent (Jacobian transpose)

Question answered: how do you find the angles for a target when there is no IK formula? For the five-bar there is one, but the method is the same for mechanisms that have none, and every answer can be checked against the exact IK.

The idea. Guess the angles, see where the pen lands, measure the miss, nudge the angles in a direction that reduces the miss, repeat.

1. Cost. Turn the miss into one number: $E = \tfrac{1}{2}\cdot|e|^2$ with $e = \text{target} - \text{FK}(\theta)$. E is zero only at the answer and grows the further off you are. Picture it as a landscape over the (θ1, θ2) plane; the answer is the bottom of the bowl.
2. Slope. E depends on θ through FK, so by the chain rule $dE/d\theta = (dE/dP)\cdot(dP/d\theta) = (-e)\cdot J = -J^T\cdot e$. The Jacobian appears because $dP/d\theta$ is exactly J. The transpose is not a trick: it is what the chain rule produces once the shapes line up (see below).
3. Update. Walk against the slope: $\theta \leftarrow \theta - \alpha\cdot dE/d\theta = \theta + \alpha\cdot J^T\cdot e$. α is the step size. In the code the result of J^Te is in radians (J is mm/rad), so convert to degrees before adding to θ.
4. Stop when $|e|$ is below a tolerance (0.01 mm is plenty for a pen), or after a fixed number of iterations.

Why the transpose. e is indexed by coordinate (x, y). J has rows indexed by coordinate and columns indexed by servo. J·e would give a result indexed by coordinate, which cannot be added to (θ1, θ2). $J^T\cdot e$ takes each column of J (what one servo does to the pen) and dots it with e: entry 1 is "does turning servo 1 move the pen toward the target, and how much?", entry 2 the same for servo 2. That is indexed by servo, so it can be added to θ. Each entry is the derivative of E with respect to that servo.



The miss shrinks by a roughly constant factor each step: linear convergence, the signature of gradient descent.

Step size. For this geometry α between 1e-4 and 3e-4 works; 5e-5 crawls, 6e-4 bounces and never converges. The limit comes from the steepness of the bowl (roughly 2 divided by the largest eigenvalue of J^TJ). In practice try values, or let the method adapt (line search: halve α whenever E goes up).

How it compares. Newton's method uses the same loop with a different direction line, $\Delta\theta = J^{-1} \cdot e$: it needs the inverse, has no α to tune, and converges much faster near the answer (the miss roughly squares each step) but explodes where J^{-1} does. Gradient descent needs only the transpose, is slow near the answer, and stalls where J goes to zero. The two fail at opposite singularities: Newton at type 1 ($J \to 0$), gradient descent at type 2 ($J \to \infty$). Damped least squares, $\Delta\theta = (J^TJ + \lambda I)^{-1} \cdot J^T \cdot e$, sits between them ($\lambda = 0$ is Newton, large λ is gradient descent) and behaves at both. For a real controller that is the one to use.

Two things this solver does not know. First, which branch: started from a crossed (elbow-in) guess it converges to a pose the plastic cannot reach. Seed it with the previous pose and cap the step size so it never crosses the D = 0 line. Second, the physical restrictions: it only sees the error and its slope. Servo range (0–180), the no-crossing rule, the trap line, the loop closing at all, none of it is in J. These rules have to be added by hand, either as a feasibility check that rejects or shrinks a step, or as extra penalty terms in the cost that make the forbidden poses uphill so the solver rolls away from them.

8. What carries over to a delta robot

A delta robot is this mechanism's big brother: three loops instead of one, in 3D. Almost everything above transfers unchanged; only the geometry gets a third dimension.

Five-bar	Delta
2 servos, 2 short links, 2 long links, 1 pen	3 servos, 3 upper arms, 3 parallelogram forearms, 1 platform
FK: pen = crossing of 2 circles (radius b) around the elbows	FK: platform = crossing of 3 spheres around the elbows
IK: each side is an independent 2-link chain, law of cosines	IK: each arm is an independent chain in its own vertical plane, same law of cosines
Constraint: $\|P - Bi\|^2 = b^2$, i = 1, 2	Constraint: $\|P - Bi\|^2 = b^2$, i = 1, 2, 3

LHS: 2 rows, the long-link arrows	LHS: 3 rows, the forearm arrows
diag(c1, c2): transmission numbers	diag(c1, c2, c3)
$J = LHS^{-1} \cdot diag(c)$, 2×2	$J = LHS^{-1} \cdot diag(c)$, 3×3
Type 1: a side straight, ci = 0	Type 1: an arm straight, ci = 0
Type 2: long links in a line, det(LHS) = 0	Type 2: forearms coplanar / parallel, det(LHS) = 0, platform can be pushed with motors locked
Servo speeds: ωi = (Ri·v) / ci	Same, three projections
Solver loop, thresholds, feasibility checks	Identical; FK often done with the Newton loop (sphere intersection) in practice

New things a delta adds: 3D vectors (cross products, planes), a dome-shaped workspace instead of a disc, and the parallelogram forearms, which keep the platform from rotating. The way of thinking, differentiate the constraint, get two matrices, watch two determinants, is exactly the same.

9. Symbols at a glance

Symbol	Meaning	Type
d, a, b	servo spacing 25.8, short link 60, long link 70 (mm)	Numbers
A1, A2	servo shafts, fixed at (∓12.9, 0)	Points
θ1, θ2	short-link angles, CCW from +x, set by the servos	degrees
B1, B2	elbows, free pins	points
P	pen	point
V, L, M, h, n	FK helpers: elbow-to-elbow arrow, its length, midpoint, triangle height, unit normal	–
D (in IK)	servo-to-pen distance for one side	mm
φ, α	IK: direction to the pen, opening angle from the law of cosines	degrees
R1, R2	long-link arrows P − B1, P − B2 (length 70)	arrows
U1, U2	elbow velocities, $a \cdot \omega \cdot (-\sin\theta, \cos\theta)$	mm/s
ω1, ω2	servo turning speeds	rad/s
v	pen velocity	mm/s
c1, c2	transmission numbers $a \cdot [R \cdot (-\sin\theta, \cos\theta)]$	mm²
LHS	matrix with rows R1, R2	2×2
D = det(LHS)	$70 \cdot 70 \cdot \sin$(angle between long links); zero at type 2	mm²
J	$LHS^{-1} \cdot diag(c1, c2)$; v = J·ω; det(J) = c1c2/D; zero at type 1	mm/rad
e, E	solver miss target − P, and cost $\frac{1}{2}\lvert e \rvert^2$	mm, mm²
α (solver)	gradient-descent step size	–